

RNA-SEQ PIPELINE

Master Runbook

PRJNA229998 — Airway Smooth Muscle Glucocorticoid-Response Study

Project	PRJNA229998 Airway RNA-Seq Pipeline (Himes et al. 2014 / GSE52778)
Environment	Slurm-managed HPC cluster
Curriculum	MIBO8110 Applied Omics — Comprehensive Bulk RNA-seq module
Document type	Operational Standard Operating Procedure (SOP) Runbook
Companion document	README.md (repository overview, architecture, citation)

Generated for internal course and reproducibility use.

0. Document Control

Field	Value
Scope	Execution and troubleshooting of the PRJNA229998 airway RNA-seq pipeline, scripts 1–9
Audience	Students and instructors running the pipeline on a Slurm HPC cluster
Companion doc	README.md (architecture, citation, license, dataset background)
Verification status	Reviewed line-by-line and syntax-checked (bash -n); NOT executed end-to-end in the authoring environment (no HPC access from this workstation) — see §10

1. Purpose & Scope

This Runbook is the step-by-step operational companion to the repository's README.md. Where the README explains what the pipeline is and why it is built the way it is, this document tells an operator exactly what to type, what to check after each step, and what to do when a step fails. It assumes the reader already has a Slurm account on the target HPC cluster with a writable `/scratch` allocation.

The pipeline takes raw SRA reads for four samples from BioProject PRJNA229998 (Himes et al. 2014, GSE52778 — a dexamethasone treatment-vs-control airway smooth muscle cell study) and produces a gene-level count matrix suitable for import into DESeq2.

Read this before your first production run

Section 10 (“Known Limitations”) documents specific, verified issues found in this repository's scripts during review — including one that can silently defeat Script 8's per-sample fault tolerance. Read it before submitting an unattended cohort run.

2. Study & Dataset Background

Field	Value
Study	Himes BE, et al. (2014). RNA-Seq Transcriptome Profiling Identifies CRISPLD2 as a Glucocorticoid Responsive Gene that Modulates Cytokine Function in Airway Smooth Muscle Cells. PLOS ONE.
DOI	10.1371/journal.pone.0099625
GEO series	GSE52778
BioProject	PRJNA229998
Design	Paired design: donor cell line × dexamethasone treatment (treated vs. untreated)
Samples processed	SRR1039508, SRR1039509, SRR1039512, SRR1039513
Reference genome	Ensembl GRCh38, release 110 (primary assembly FASTA + GTF)

This dataset is also the canonical worked example behind the Bioconductor DESeq2 vignette and the pre-built airway package — useful for having students cross-check their own recount against a trusted reference count matrix. SRR1039513 is additionally a useful, real edge case for teaching read-pair integrity: it contains over 26.5 million zero-length reads, which is what originally broke an earlier version of this pipeline (see §9).

3. Pipeline Architecture

3.1 Two-tier design

- Modular suite (scripts 1, 2, 3, 4, 5, 6, 7, 9) — single-sample (SRR1039508) walkthrough, one Slurm job per stage, workspace `/scratch/$(whoami)/PRJNA229998_airway_pipeline_stepbystep/`.
- V9 automated cohort suite (script 8) — all four samples in one fault-tolerant Slurm job, workspace `/scratch/$(whoami)/PRJNA229998_airway_pipeline_automated/`.

Both tiers share one reference directory, `/scratch/$(whoami)/PRJNA229998_airway_pipeline_ref/`, holding the downloaded Ensembl FASTA/GTF and the built STAR index. That index is built by Script 6 the first time it runs — Script 8 assumes it already exists and does not build it (see §6).

3.2 Historical / reference-only files

`PRJNA229998_airway_pipeline.sh` and `Final_airway_pipeline_v9_standalone.sh` are earlier, single-file monolithic drafts kept for historical reference. They are not the recommended execution path — `PRJNA229998_airway_pipeline.sh` in particular still uses `fasterq-dump --split-files` rather than `--split-3`, which will reproduce the SRR1039513 read-desynchronization failure described in §9. Use scripts 1–9.

4. Prerequisites

4.1 Environment modules

Tool	Module (as used in these scripts)
SRA Toolkit	SRA-Toolkit/3.0.3-gompi-2022a
FastQC	FastQC/0.11.9-Java-11
fastp	fastp/0.23.4-GCC-13.2.0
STAR	STAR/2.7.10b-GCC-11.3.0
MultiQC	MultiQC/1.28-foss-2024a
R (Bioconductor-capable)	R (cluster default — confirm Rsubread availability, see §10.2)

Module names and versions are cluster-specific. Run `module avail <tool>` and edit the `module load` lines at the top of each script if your cluster's naming differs — this is exactly the failure documented in §9.2 (Environment & Dependency Management).

4.2 Compute resources

- Scratch space: budget ~35–40 GB for the shared GRCh38 FASTA + GTF + STAR index, plus several GB per sample for raw/trimmed FASTQs and BAMs.
- Memory: STAR's `genomeGenerate` step needs ~31.5 GB resident memory. Never attempt this on a shared login node — request a dedicated 36 GB Slurm allocation (see §9.6).
- Network egress from compute nodes is required (SRA prefetch, Ensembl FTP downloads in Script 6).

5. Directory Structure & Data Flow

```
/scratch/$(whoami)/
├─ PRJNA229998_airway_pipeline_ref/          # Shared: GRCh38 FASTA/GTF + STAR index
├─ PRJNA229998_airway_pipeline_stepbystep/   # Modular workspace (SRR1039508 only)
│   ├─ raw_reads/  checksums/  qc_before/  trimmed_reads/
│   ├─ fastp_reports/  qc_after/  alignments/  counts/
│   └─ logs/  sra_cache/  FINAL_PIPELINE_REPORT.md
└─ PRJNA229998_airway_pipeline_automated/    # V9 cohort workspace (4 samples)
    ├─ raw_reads/  checksums/  qc_before/  trimmed_reads/
    ├─ fastp_reports/  pairing_reports/  alignments/
    └─ logs/  sra_cache/  counts/airway_raw_counts.txt(.summary)
```

6. Standard Operating Procedure — Modular Walkthrough

Run once, in strict order, on SRR1039508. Wait for each Slurm job to complete (`squeue -u $(whoami)` shows nothing for your job) before submitting the next — there is no automatic `--dependency` chaining between these scripts.

Step 1 — Fetch & Verify Raw Reads

```
sbatch 1_Fetch_And_Verify_Raw_Reads.sh
```

- What it does: prefetch → vdb-validate (cache-corruption gate) → fasterq-dump --split-3 → md5sum baseline.
- Resources: 4 CPU, 8 GB RAM, 1 h wall time.
- Verify: `raw_reads/SRR1039508_1.fastq` and `_2.fastq` exist and are non-empty; `checksums/SRR1039508_raw.md5` has two lines.
- If it fails: check `logs/SRR1039508_fetch.log`. A vdb-validate failure means a corrupted SRA cache — delete `sra_cache/SRR1039508/` and resubmit.

Step 2 — Interpret FastQC Diagnostics

```
sbatch 2_Interpret_FastQC_Diagnostics.sh
```

- What it does: FastQC on both raw mates with `--extract`, then parses `summary.txt` into PASS/WARN/FAIL counts printed to the Slurm log.
- Resources: 4 CPU, 6 GB RAM, 45 min.
- Verify: `qc_before/SRR1039508_1_fastqc/summary.txt` and the `_2` equivalent both exist. Note the WARN/FAIL count — Script 4 reads these files directly.

Step 3 — Batch MultiQC Summary

```
sbatch 3_Batch_MultiQC_Summary.sh
```

- What it does: aggregates `qc_before/` into one interactive dashboard, `multiqc_raw_batch.html`.
- Resources: 4 CPU, 8 GB RAM, 1.5 h.
- Verify: `multiqc_raw_batch.html` exists at the workspace root; open it to visually confirm Script 2's parsed verdicts.

Step 4 — Evidence-Based fastp Trimming

```
sbatch 4_Evidence-Based_Fastp_Trimming.sh
```

- What it does: re-reads Script 2's FastQC summary.txt files. Only if Adapter Content was flagged WARN/FAIL does it add --detect_adapter_for_pe; only if Per base sequence quality was flagged does it add --cut_tail --cut_tail_mean_quality 20. A 36 bp minimum-length floor is always applied. Writes the decision to fastp_reports/SRR1039508_THRESHOLD_RATIONALE.md, then FastQC's the cleaned reads into qc_after/.
- Resources: 4 CPU, 8 GB RAM, 1 h.
- Verify: trimmed_reads/SRR1039508_1_clean.fastq / _2_clean.fastq exist; read fastp_reports/SRR1039508_THRESHOLD_RATIONALE.md and confirm the stated rationale matches what Step 2 actually reported.

Step 5 — Paired-End Integrity Check

```
sbatch 5_Paired_End_Integrity_Check.sh
```

- What it does: confirms mate 1 and mate 2 of the trimmed reads still have identical read counts and identical, sequentially matched read IDs (mate-suffix-stripped).
- Resources: 2 CPU, 4 GB RAM, 30 min.
- Verify: Slurm log prints "OK: Pairing intact." A "FAIL" here means Script 4's trimming desynchronized the mates — stop and investigate before aligning; do not proceed to Step 6.

Step 6 — STAR Alignment

```
sbatch 6_STAR_Alignment.sh
```

- What it does: downloads Ensembl GRCh38 FASTA/GTF into the shared reference directory if absent, builds the STAR index if absent (~31.5 GB RAM, several hours the first time only), then aligns SRR1039508 to a coordinate-sorted BAM.
- Resources: 8 CPU, 36 GB RAM, 2 h. This step is why 36 GB is requested — see §9.6 for what happens if you run this on a login node instead.
- Verify: alignments/SRR1039508_Aligned.sortedByCoord.out.bam exists; tail -n 20 alignments/SRR1039508_Log.final.out and confirm a sane "Uniquely mapped reads %" (typically >85% for a clean human RNA-seq sample against GRCh38).
- This step also produces the shared STAR index other jobs (including Script 8) depend on — do not delete PRJNA229998_airway_pipeline_ref/star_index/ between runs.

Step 7 — Quantification (Rsubread)

```
sbatch 7_Quantification_Rsubread.sh
```

- What it does: writes and runs a small R script calling `Rsubread::featureCounts()` on the BAM from Step 6. Bootstraps Rsubread into a local `R_libs/` scratch directory via BiocManager if the cluster's R module doesn't already provide it.
- Resources: 4 CPU, 16 GB RAM, 1 h. First run may take longer if it needs to compile/install Rsubread.
- Verify: `counts/SRR1039508_counts.txt` (gene × sample matrix) and `counts/SRR1039508_counts.txt.summary` (assignment breakdown) both exist; check the “Assigned” row's proportion of total reads.

Step 9 — QC Report Generator

```
bash 9_QC_Report_Generator.sh # run locally, NOT via sbatch
```

- What it does: reads the raw/trimmed read counts, the Step 5 pairing verdict, the Step 6 STAR mapping rate, and the Step 7 assignment count, and assembles them into `FINAL_PIPELINE_REPORT.md`.
- Resources: 1 CPU, 4 GB RAM, 10 min — lightweight enough to run interactively on the login node.
- Verify: `FINAL_PIPELINE_REPORT.md` exists at the workspace root and every field is populated (not “N/A”). An “N/A” value means an earlier step's expected output file is missing — go back and check that step.

Scope note

There is no Script 8 equivalent — Script 9 only ever summarizes the single-sample `_stepbystep` workspace, never the 4-sample `_automated` cohort. See §7.

7. Standard Operating Procedure — Automated Cohort Run (Script 8)

Script 8 re-implements fetch → FastQC → evidence-based fastp → pairing check → STAR alignment as one Slurm job, looped over all four samples in its own workspace, then runs one batch featureCounts across every sample that reported success.

```
sbatch 8_Automated_Pipeline_AllSamples.sh
```

- Resources: 8 CPU, 36 GB RAM, 12 h wall time (covers all four samples sequentially).

Hard prerequisite

Script 8 references `${REFDIR}/star_index` directly and does NOT build it. Run Step 6 of the modular walkthrough at least once first, so `PRJNA229998_airway_pipeline_ref/star_index/` already exists, before submitting Script 8.

Monitor per-sample progress via `logs/<SRR>_pipeline.log` inside the `_automated` workspace. On completion, check `counts/airway_raw_counts.txt` and `counts/airway_raw_counts.txt.summary` for the combined cohort matrix, and `pairing_reports/<SRR>.txt` for each sample's trimmed-read pairing verdict.

Read §10.1 and §10.2 before relying on this script unattended

Two verified issues affect this script specifically: its per-sample fault tolerance can be defeated by the outer set `-e`, and its final quantification step has no Rsubread bootstrap fallback. Both are detailed in §10.

8. Monitoring & Troubleshooting Quick Reference

```
# Queue status
squeue -u $(whoami)

# Cancel a job
scancel <JOB_ID>

# Tail a step's custom log
tail -f logs/SRR1039508_fetch.log

# Tail STAR's live progress during alignment
tail -f alignments/SRR1039508_Log.progress.out

# Sweep Slurm stdout for common failure signatures
grep -iE "error|fatal|cannot|execution halted" ~/slurm-*.out

# Confirm STAR mapping rate
tail -n 20 alignments/*_Log.final.out

# Confirm featureCounts assignment rate
cat counts/*_counts.txt.summary
```

9. Troubleshooting History (v1 → v9)

The following is distilled from TROUBLESHOOTING_REPORT.txt (v1–v6) and Detailed_Pipeline_Troubleshooting_Report.pdf (v7–v9), both included in this repository. Every fix listed in §3 of the README traces back to one of these phases.

9.1 Syntax corruption via copy-paste artifacts (v1→v4)

Symptom: Slurm aborted early script versions with a syntax error near an unexpected token, rendered as `\vert{}\vert{}`. Root cause: a web clipboard tool silently converted bash's `||` operator into LaTeX math formatting during copy-paste into the terminal. Fix (v4): eliminated `||` entirely, replaced with explicit multi-line `if/else` statements that survive that kind of corruption.

9.2 Lmod toolchain and dependency clashes (v4→v5)

Symptom: Lmod warnings about missing GCC/zlib dependent modules; fastp silently lost its required C libraries after STAR's module load forced a GCC downgrade. Root cause: loading all tools globally at the top of the script caused a dependency war between fastp's GCC/13 requirement and STAR's GCC/11 requirement. Fix (v5): just-in-time module loading — `module purge` immediately before each `module load` block, isolating each tool's runtime environment. This is the `module purge` pattern present at the top of every script 1–9.

9.3 The silent SIGPIPE trap (v5→v6)

Symptom: a job failed rapidly (exit code 1:0) immediately after a successful prefetch, with no further explanation in the log. Root cause: `vdb-validate ... | tee -a "$LOG" | grep -q "is consistent"` — `grep -q` exits the instant it finds its match, closing the pipe; `vdb-validate` then received SIGPIPE (exit 141) writing into the closed pipe, and `set -o pipefail` propagated that as a fatal error, killing the whole job. Fix (v6): removed the `grep -q` trap from the validation pipeline, and wrapped each sample's whole processing block in a subshell inside an `if` statement so one sample's failure doesn't (in principle) crash the batch — see the caveat on this in §10.1.

9.4 Read desynchronization from --split-files (v7→v8)

Symptom: fastp failed with `ERROR: input files don't contain identical amount of reads on SRR1039513`. Root cause: SRR1039513 contains over 26.5 million zero-length reads; `fasterq-dump --split-files` drops these asynchronously between mates, desynchronizing `_1.fastq/_2.fastq` line counts. Fix: re-extracted with `--split-3`, which routes unmated/zero-length reads to a separate singletons file instead, preserving 1:1 mate parity. Verified by `wc -l` matching exactly (67,292,352 lines each mate) after the fix. This is now the `--split-3` standard used throughout scripts 1 and 8.

9.5 Module capitalization and toolchain conflicts, restated (v8)

Symptom: module not found and further Lmod dependency warnings during the SRA-Toolkit → fastp transition. Root cause: the SRA module's exact name/capitalization (SRA-Toolkit, not sra-toolkit) differed from what was assumed. Fix: used `module avail sra` to find the exact name, then re-applied the module `purge` → `module load GCC/13.2.0 fastp STAR Subread` discipline from §9.2.

9.6 RAM exhaustion on the login node (v8→v9)

Symptom: STAR alignment crashed with `std::bad_alloc` while loading the GRCh38 index. Root cause: the GRCh38 STAR index requires ~31.5 GB memory; the shared interactive login node enforces a much lower quota and killed the process immediately. Fix: requested a dedicated interactive Slurm compute node instead of running directly on the login node.

Exact srun escalation observed on this cluster

```
# Attempt 1 – missing partition
srun --cpus-per-task=8 --mem=36G --time=01:00:00 --pty bash
srun: error: You must specify a partition (queue) to run in.

# Attempt 2 – missing task specification
srun -p batch --cpus-per-task=8 --mem=36G --time=01:00:00 --pty bash
srun: error: Unable to allocate resources: Job size specification needs to be provided
srun: error: You must specify a number of tasks.

# Attempt 3 – successful allocation
srun -p batch -n 1 --cpus-per-task=8 --mem=36G --time=01:00:00 --pty bash
# Prompt changes to the allocated compute node, e.g. user@rbl-5
```

This is why `--mem=36G` is hardcoded into every STAR-related `#SBATCH` header in scripts 6 and 8, and why the README explicitly warns never to run STAR indexing on a login node.

9.7 Standard Operating Procedures adopted from this history

- SRA extraction: always `--split-3`, never `--split-files`.
- Job submission: never run memory-intensive steps (STAR) on a login node; use `srun -p batch -n 1 --mem=36G` for interactive testing, `sbatch` for production.
- Environment isolation: `module purge` before every toolchain transition.

10. Known Limitations in the Current Release

These were identified by a line-by-line review of scripts 1–9 against their own documented intent and against each other — not by an actual failed run, since this documentation was authored without HPC access. They are recorded here, not silently fixed in the scripts, so the next person to run this pipeline knows exactly what to verify.

10.1 Script 8's per-sample fault tolerance can be defeated by `set -e`

Each sample in Script 8's loop runs inside `( set -e; set -o pipefail; ... )`, and the outer script itself also runs under `set -e` (near the top of the file). Because the subshell is invoked as an ordinary top-level command — not inside an `if`, `&&`, or `||` — a non-zero exit from any single sample's subshell triggers the outer script's `set -e` and terminates the whole Slurm job immediately, before the `if [ $? -eq 0 ]` line that is meant to route the failure into `FAILED_SRRS` is ever reached.

Practical impact

One bad sample can still abort the entire cohort run — exactly the failure mode the subshell wrapping was designed to prevent (per §9.3's fix). The same pattern exists in the legacy `PRJNA229998_airway_pipeline.sh`.

Workaround until the script is patched: wrap the subshell dispatch in a context `set -e` will not fire on — for example, `if ( set -e; ...; ); then ...; fi` — or temporarily `set +e` around the loop body and re-enable `set -e` after capturing `$?` .

10.2 Script 8's final quantification step has no Rsubread bootstrap

Script 7's R script installs Rsubread into a local `R_libs/` directory via BiocManager if it isn't already available. Script 8's `counts/run_final_quant.R` does not — it calls `suppressMessages(library(Rsubread))` directly. If the cluster's default R module lacks Rsubread, this fails only after all four samples have already been fetched, trimmed, and aligned — the most expensive part of the run.

Mitigation: before submitting Script 8, confirm `Rscript -e 'library(Rsubread)'` succeeds under the R module Script 8 loads, or point `R_LIBS_USER` at the `R_libs/` directory Script 7 already populated in the sibling `_stepbystep` workspace.

10.3 `PRJNA229998_airway_pipeline.sh`'s self-assessment does not match its own code

This legacy monolithic script still calls `fasterq-dump --split-files` (not `--split-3`), yet carries an appended “Assessment of V9” comment block claiming the `--split-3` flag is hardcoded. That claim is

accurate for `Final_airway_pipeline_v9_standalone.sh`, not for this file — the two were evidently not kept in sync. Run as-is against SRR1039513, this script would reproduce the exact read-desynchronization failure documented in §9.4.

10.4 --sjdbOverhang is inconsistent across the three pipeline variants

File	--sjdbOverhang value
6_STAR_Alignment.sh (current modular suite)	74
Final_airway_pipeline_v9_standalone.sh	99
PRJNA229998_airway_pipeline.sh	100

STAR's own guidance is read length – 1. Confirm actual FASTQ read length before a from-scratch index build (e.g., `awk 'NR==2{print length($0); exit}' raw_reads/<SRR>_1.fastq`) rather than assuming 74 is correct for your data — it is simply the most recently maintained value in this repository, not an independently re-derived one.

10.5 Script 9 does not cover the automated cohort workspace

`9_QC_Report_Generator.sh` only ever reads from the `_stepbystep` workspace and only ever reports on SRR1039508. It is not wired up to summarize the 4-sample `_automated` workspace Script 8 writes to. For a cohort-level summary today, review each sample's `logs/<SRR>_pipeline.log` and the combined `counts/airway_raw_counts.txt.summary` directly.

11. Downstream Handoff to DESeq2

Terminal outputs, tab-delimited, gene × sample:

- Modular suite (single sample): `counts/SRR1039508_counts.txt`
- Automated cohort: `counts/airway_raw_counts.txt` (all passed samples as columns)

Both are import-ready for `DESeqDataSetFromMatrix()` in R. Because GSE52778 is the canonical DESeq2/airway-package worked example, the `colData` design matrix (`cell_line`, `dex` treatment/untreated) can be built directly from the study's public SRA RunInfo metadata, and a student's own recount can be sanity-checked against the pre-built Bioconductor airway object.

Appendix A — Resource Allocation Reference

Script	CPUs	Memory	Wall time
1_Fetch_And_Verify_Raw_Reads.sh	4	8 GB	1 h
2_Interpret_FastQC_Diagnostics.sh	4	6 GB	45 min
3_Batch_MultiQC_Summary.sh	4	8 GB	1.5 h
4_Evidence_Based_Fastp_Trimming.sh	4	8 GB	1 h
5_Paired_End_Integrity_Check.sh	2	4 GB	30 min
6_STAR_Alignment.sh	8	36 GB	2 h
7_Quantification_Rsubread.sh	4	16 GB	1 h
8_Automated_Pipeline_AllSamples.sh	8	36 GB	12 h
9_QC_Report_Generator.sh	1 (local)	4 GB	10 min

Appendix B — Full Command Reference

```
# Modular walkthrough (single sample, run in order, wait between jobs)
sbatch 1_Fetch_And_Verify_Raw_Reads.sh
sbatch 2_Interpret_FastQC_Diagnostics.sh
sbatch 3_Batch_MultiQC_Summary.sh
sbatch 4_Evidence_Based_Fastp_Trimming.sh
sbatch 5_Paired_End_Integrity_Check.sh
sbatch 6_STAR_Alignment.sh
sbatch 7_Quantification_Rsubread.sh
bash 9_QC_Report_Generator.sh

# Automated cohort (all 4 samples) – only after Script 6 has built the shared index once
sbatch 8_Automated_Pipeline_AllSamples.sh
```

Appendix C — Version Evolution Summary

Version	Change
v1	Initial draft; failed due to <code> </code> → LaTeX copy-paste corruption.
v2	Attempted inline sed cleanup of the syntax corruption.
v3	Cleaned bracket logic; still vulnerable to the same web-formatting artifact.
v4	Fully eliminated <code> </code> operators — permanent fix for the syntax corruption.
v5	Introduced just-in-time <code>module purge / module load</code> to resolve GCC toolchain clashes.
v6	Resolved the <code>vdb-validate grep -q SIGPIPE</code> trap; wrapped per-sample execution in a subshell + <code>if</code> .
v7–v8	Diagnosed and fixed SRR1039513 read desynchronization (<code>--split-files</code> → <code>--split-3</code>); resolved module capitalization issues.
v9	Resolved STAR <code>std::bad_alloc</code> via dedicated 36 GB compute-node allocation; validated fixes cascaded into scripts 1–9.

Citation

Himes BE, Jiang X, Wagner P, Hu R, Wang Q, Klanderman B, et al. (2014) RNA-Seq Transcriptome Profiling Identifies CRISPLD2 as a Glucocorticoid Responsive Gene that Modulates Cytokine Function in Airway Smooth Muscle Cells. *PLoS ONE* 9(6): e99625. <https://doi.org/10.1371/journal.pone.0099625>